Object-Oriented Concepts with UML
Classes & Structure

CCP6224

Object-oriented Analysis and Design

Lecture outcomes

- At the end of this lecture, you should
  - Understand how classes work.
  - Be able to draw the UML diagrams for classes.
  - Know how to create instances from classes using constructors.
  - Know how to access variables and methods of classes.
  - Deal with access specifiers (privacy).
  - Use getters and setters.
  - Understand the difference between class and instance variables.
  - Know how to use data promotion and casting.

Object classes

- A class is a *template* for object creation
- Classes may have fields/attributes and methods/functions – each with access specifiers
- Instances are objects created (instantiated) from a class
- Each object instance has a state defined by its attributes

- In Java, every program must have at least one class (as opposed to C++)

A class
(the concept)

An object
(the realization)

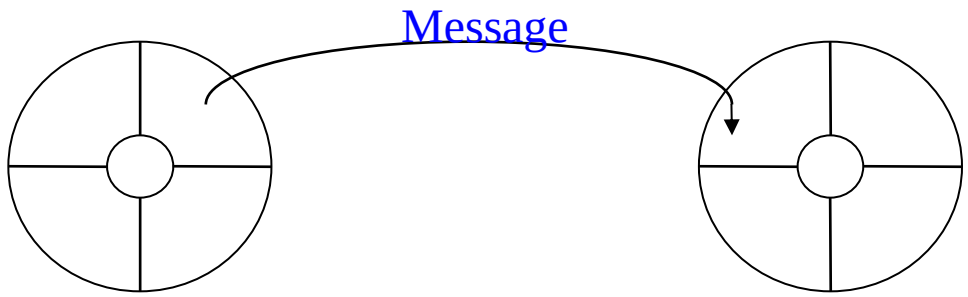
John's Bank Account
Balance: $5,257

Bill's Bank Account
Balance: $1,245,069

Multiple objects
from the same class

Mary's Bank Account
Balance: $16,833

- Objects interact with each other through methods by sending and receiving messages.
- The methods (functions) implement the behaviour of the object.
- Methods may be overloaded and/or overridden.
- The class encapsulates the data and methods and provides methods to prevent uncontrolled access.
- Once declared, classes are treated as a new data type




```
String s = "Hi, world!";

int n = s.length();
```

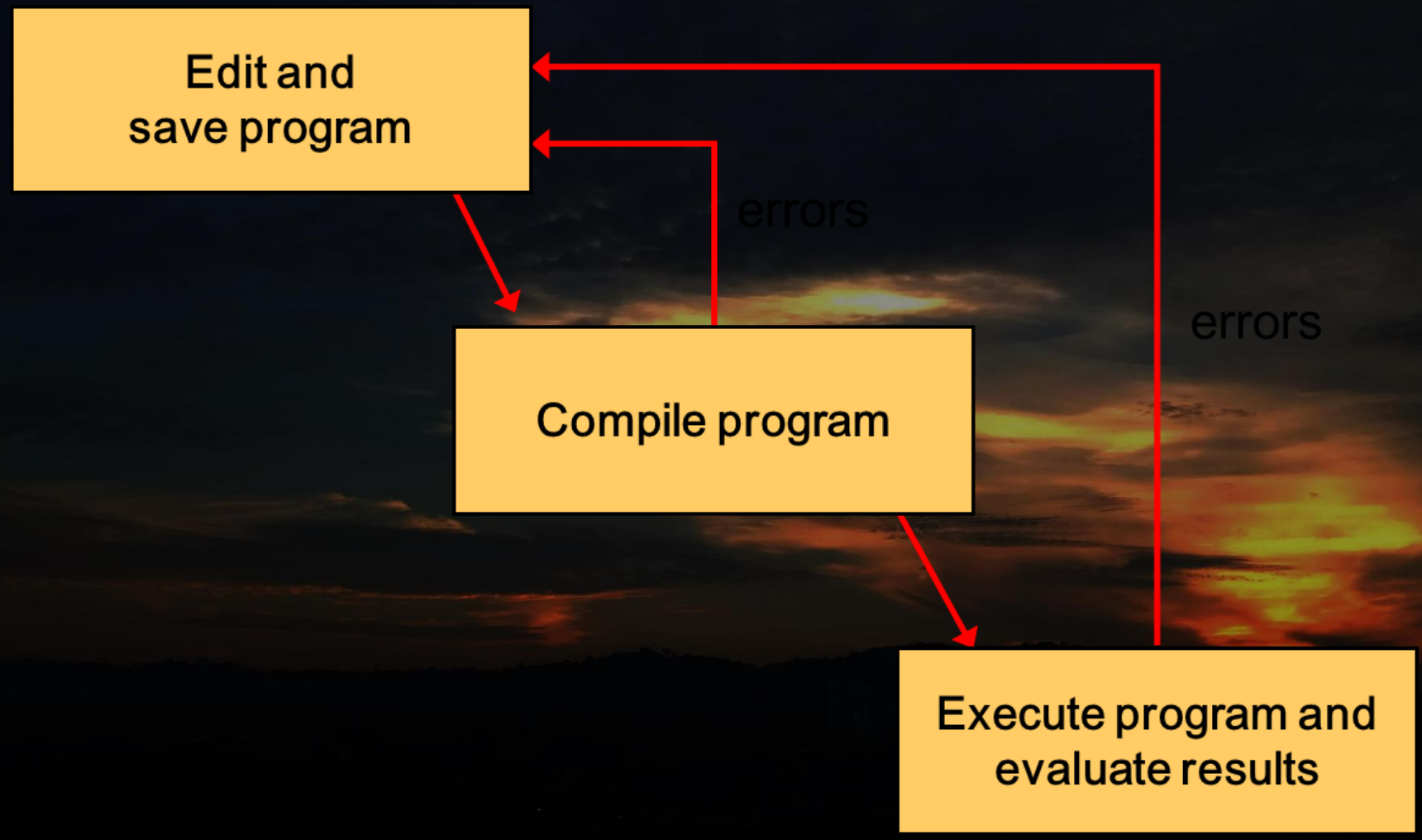
Java classes

- In the Java programming language:
  - A program is made up of one or more classes
  - A class contains one or more methods
  - A method contains program statements
- These terms will be explored in detail throughout the course through Java applications

- A Java *application* MUST always contains a method called main

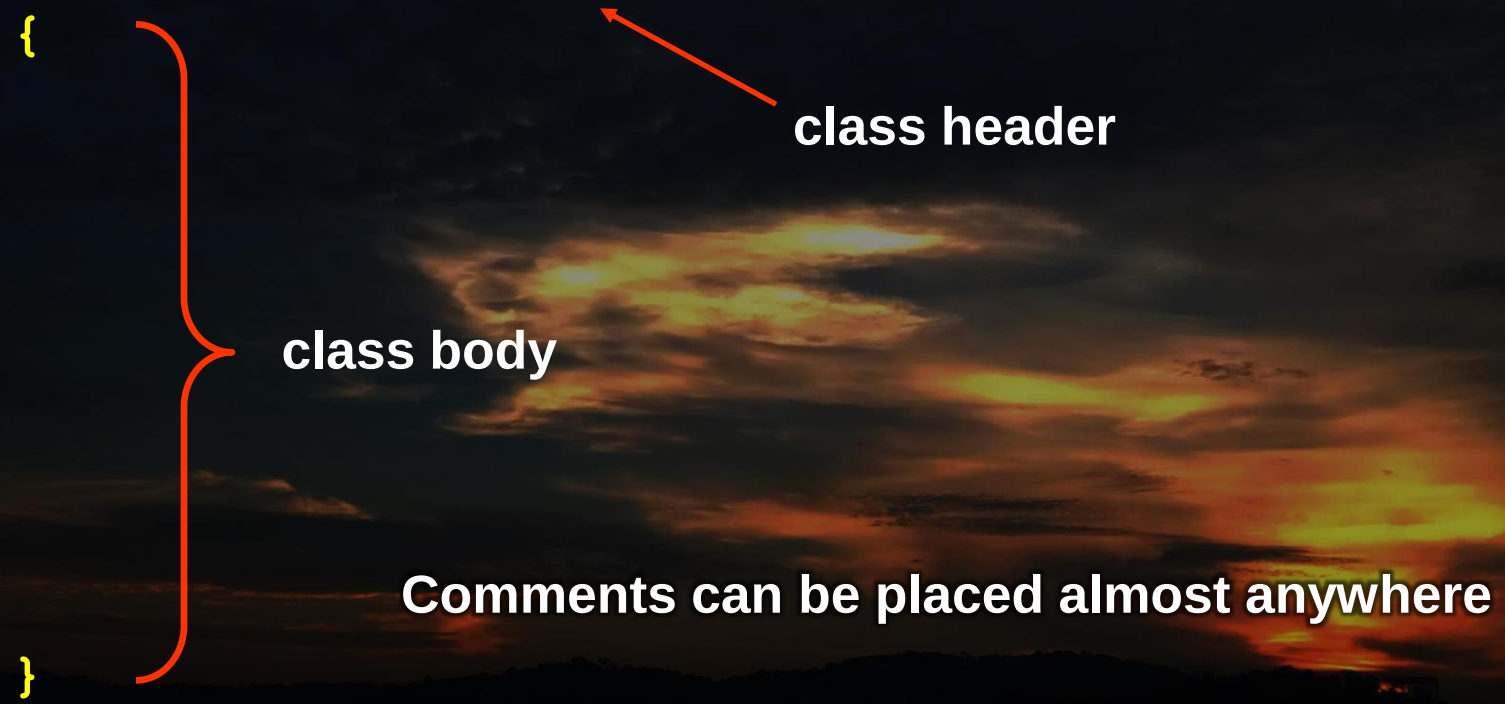
- Common error → classes have no parentheses ‘( )’ because they are not functions!!!

Java program development



Java application program structure

```
//  comments about the class
public class MyProgram
{


}
```




// comments about the class
public class MyProgram
{
// comments about the method
public static void main (String[] args)
{
}
}
method body
method header

Java Identifiers

- Basically names that programmers use to identify items of interest
- An identifier can be made up of letters, digits, the underscore character (_), and the dollar sign
- Identifiers cannot begin with a digit
- Java is case sensitive - `Total`, `total`, and `TOTAL` are different identifiers
- Convention: names are camel-cased beginning with capitals in class names (e.g. `StudAcct`) and small letters in methods and variables (e.g. `checkBal`)
- Often we use special identifiers called reserved words that already have a predefined meaning in the language
- A reserved word cannot be used in any other way

Java reserved words

abstract	else	interface	switch
assert	enum	long	synchronized
boolean	extends	native	this
break	false	new	throw
byte	final	null	throws
case	finally	package	transient
catch	float	private	true
char	for	protected	try
class	goto	public	void
const	if	return	volatile
continue	implements	short	while
default	import	static	
do	instanceof	strictfp	
double	int	super	

Simple Java application

```
// This program prints
// Hello World to the standard output.
class Hi {
      public static void main(String[] args){
        String x = new String("Hello World");
        System.out.println( x );
      }
}
```

The main method

- Java applications require that **`main`** MUST be defined with the attributes public AND static.
- Return type of **`main`** is usually void – returns nothing
- **`main`** expects one argument, a reference to an array that can hold references to strings passed as argument parameters

```
public static void main (String[] args)
```

Variable declaration

- A variable must be declared by specifying the variable's name and the type of information that it will hold - Multiple variables can be created in one declaration
- A variable can be given an initial value in the declaration


```
int total;
int count, temp, result;
```

- Can represent different types
  - Four represent integers: `byte, short, int, long`
  - Floating point numbers: `float, double`
  - Characters: `char, String`
  - Boolean values: `boolean`

Booleans

- A boolean value represents a true or false condition
- The reserved words **`true`** and **`false`** are the only valid values for a boolean type

```
boolean done = false;
```

- A boolean variable can also be used to represent any two states, such as a light bulb being on or off

String

- Represents a collection of characters with particular meaning
- Fundamentally **`String`** is an array of characters but is actually a new object/data type – has its own built-in functions to ensure operations work properly
- A string of characters can be represented as a string literal by putting double quotes around the text. Examples:

```
"This is a string literal."
"123 Main Street"
"X"
```

- Every character string is an object in Java, defined by the **`String`** class
- Strings can also sometimes work with array methods (but best avoided)
- Strings can be combined with other strings through concantenation (overloaded '+' operator)

Arrays in Java

- How are arrays declared in C++?

```
int foo[5] = {1,2,3,4,5};
```

```
int bar[3];
bar[0] = 50;
```

- In Java, the brackets are placed near the data type to indicate what type of array is being created

```
int[] foo = {1,2,3,4,5};
```

```
int[] bar = new int[3];
bar[0] = 50;
```

- So what does the parameter below indicate?

```
public static void main (String[] args)
```

In this case, the array can hold references to *String* objects, which are any command-line arguments passed to the program.

Printing to screen

- Output to command line screen uses built in system function from Java.
- **`System`** is a standard Java class that represents the system on which the program executes.
- **`out`** is a static field in the **`System`** class and its type is **`PrintStream`**, another standard Java class.
- **`println`** is a **`PrintStream`** method in the **`System`** class

```
System.out.println( x );
```

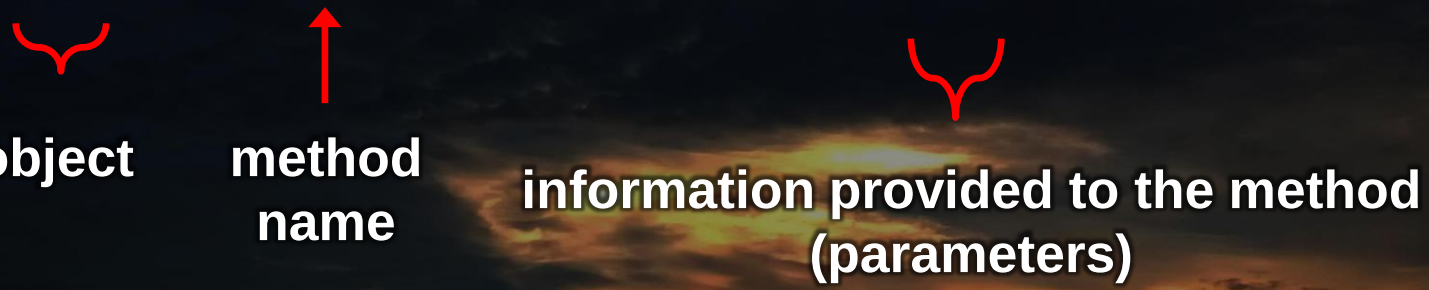
C equivalent

```
printf(x);
```

- Note the case of **`System`** and the 'l' in **`println`** is the small letter L and not number 1
- Escape characters from C++ also work in Java (e.g. **`\n`**, **`\t`** etc)

- The **`System.out`** object represents a destination (the monitor screen) to which we can send output.

```
System.out.println("Whatever you are, be a good one");
```

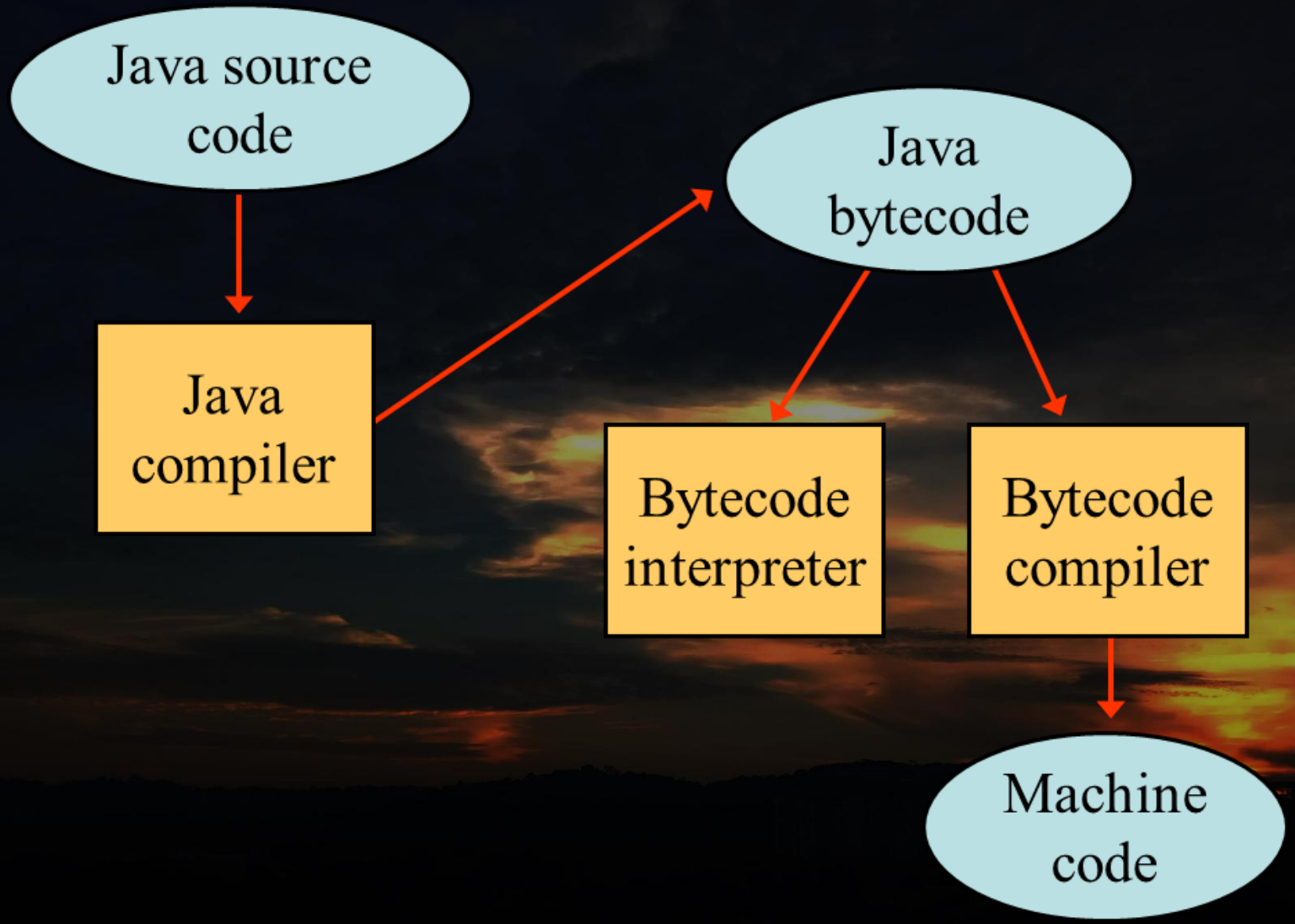


- The **`print`** method is similar to the **`println`** method, except that it does not advance to the next line
- Therefore anything printed after a **`print`** statement will appear on the same line

- What is the output of the program below when executed?

```
public class Countdown
{
   public static void main (String[] args)
   {
      System.out.print ("Three... ");
      System.out.print ("Two... ");
      System.out.print ("One... ");
      System.out.print ("Zero... ");
      System.out.println ("Liftoff!");
      System.out.println ("Houston, we have a problem");
   }
}
```

Java translation



Java compilation and execution

- Java compilation makes use of the compiler javac.
- If your program code is saved as Hi.java (must match class name of the code inside), then to compile the code the command statement is

  `javac Hi.java` ← case sensitive filenames!!!

- Compiling a source file such as Hi.java generates one or more binary files (Java Byte Code) with a .class extension, e.g., Hi.class.

- To execute the binary, load it into the JVM with the statement

  `java Hi`

Java errors

- A program can have three types of errors
  - The compiler will find syntax errors and other basic problems (compile-time errors)
  - If compile-time errors exist, an executable version of the program is not created → no binary files created
  - A problem can occur during program execution, such as trying to divide by zero, which causes a program to terminate abnormally (run-time errors)
  - A program may run, but produce incorrect results, perhaps using an incorrect formula (logical errors)

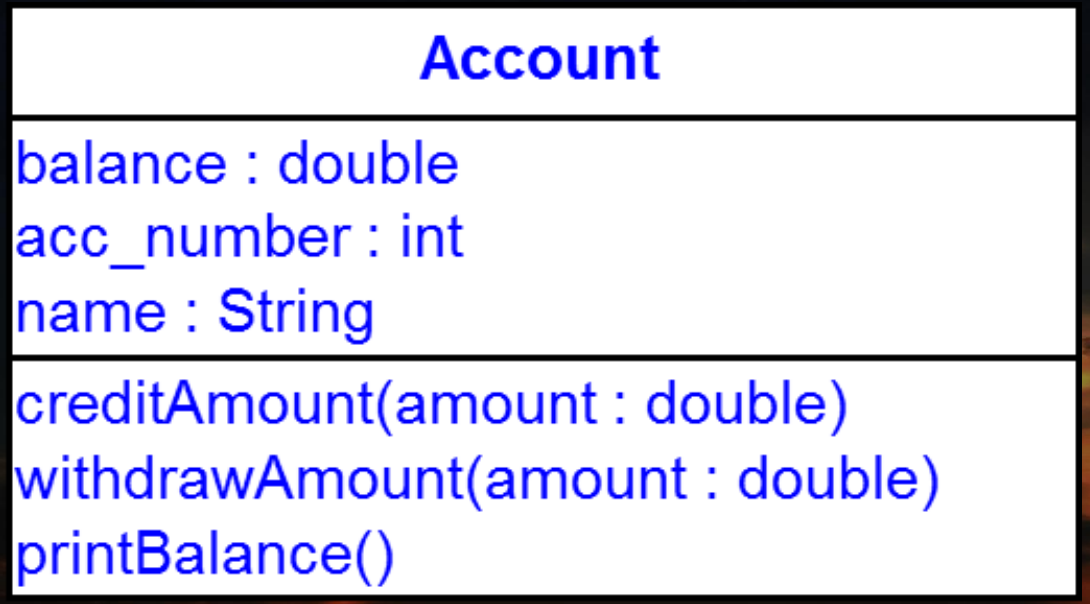
A Java object class

- Identify class name, class modifier, variables (attributes) and methods (behaviours/functions) in the code below
- This code will not compile into an application….. Why?

```
public class Account{

   double balance;
   int acc_number
   String name;

  void creditAmount(double amount){
     balance = balance+amount;
   }

 void withdrawAmount(double amount){
      balance = balance-amount;
   }

   void printBalance(){
     System.out.println
        ("Balance amount in Account No: "
        + acc_number + "  is  RM " + balance);
   }
}
```

UML representation

- You will learn UML diagrams in detail later but this is the Account class code in a basic class diagram



Object class instances

- To create an object from a class (instantiate the class), the class is treated just like a basic data type (i.e. similar to int, char, float etc)

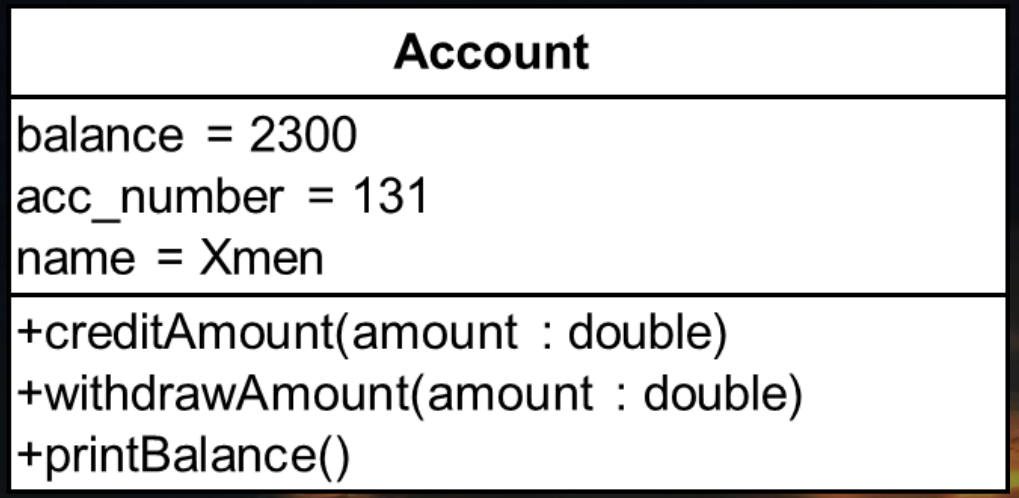
```
Account newAcct;    // similar to "int myInt;"
newAcct = new Account();
```

OR

```
Account newAcct2 = new Account();
```

Dot operator

- To access an object's fields and methods outside of the class, use objects' instance name with the dot operator



- How do you access the balance if the object's instance name is acc1234? (assume public access)
- How do you access the name if the object's instance name is acc1234? (assume public access)

`this` keyword operator

- Within an instance method or a constructor, *this* is a reference to the current object — the object whose method or constructor is being called.
- One of the main usage of keyword this is to resolve ambiguity
- You can refer to any member of the current object from within an instance method or a constructor with **`this`**

```
public class Point {
    public int x = 0;
    public int y = 0;

    //constructor
    public Point(int a, int b) {
        x = a;
        y = b;
    }
}
```

```
public class Point {
    public int x = 0;
    public int y = 0;

    //constructor
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

Access specifiers

- Java provides a number of access modifiers to set access levels for classes, variables, methods and constructors. The four access levels are:
  - Visible to the *package* – default.
    - No modifiers are needed.
  - Visible to the class only (private).
    - Most restrictive and secure
  - Visible to the world (public).
    - Least restrictive and least secure
  - Visible to the package and all subclasses (protected).
- Unlike C++, access specifiers in Java are on a per-line basis

Java packages

- Similar to C++ namespaces
- To ‘group’ related content together to ensure consistent naming, avoid name conflicts, control access, make searching/locating classes easier.
- Code can be written in several files and grouped together with the statement above (i.e. similar to header declaration)

```
package pkgname;
```

- The **package** statement should be the first line in the source file.
- There can be only one package statement in each source file, and it applies to all types in the file
- It is common practice to use lowercased names of packages to avoid any conflicts with the names of classes, interfaces

- Without package statement, files that ‘work’ together will be in the same package

Java application 2

```
// save as Account.java

public class Account{

   double balance;
   int acc_number;
   String name;

   void creditAmount(double
amount){...}

   void withdrawAmount(double
amount){...}

   void printBalance(){…}

}
```

```
// save as MyProg.java

class MyProg{

  public static void main
(String [] args) {

  Account a1 = new Account();

  a1.balance = 2300;
  a1.acc_number = 131;
  a1.name = "Xmen";

  a1.creditAmount(200);
  a1.withdrawAmount(500);
  a1.printBalance(); // 2000
}
```

MyProg works in the same ‘group’ with Account so the default specifier is used (public to package)

- Will this code compile and run? Why or why not?

```
// saved as Account.java

public class Account{

  private double balance;
  private int acc_number;
  private String name;

  void creditAmount(double
amount){...}

  void withdrawAmount(double
amount){...}

  void printBalance(){…}

}
```

```
// saved as MyProg.java

class MyProg{

  public static void main
(String [] args) {

  Account a1 = new Account();

  a1.balance = 2300;
  a1.acc_number = 131;
  a1.name = "Xmen";

  a1.creditAmount(200);
  a1.withdrawAmount(500);
  a1.printBalance(); // 2000
}
```

Getters and Setters

- The most basic methods in a class
- Getters get the value of inaccessible variables whilst setters set the value of inaccessible variables ….. But WHY?!?!?

Why not set everything public

- Although leaving variables as public is easier, it allows any object/method to change the value
- By using a get/set method, any changes made are controlled because they can only be altered/retrieved by the class's methods and additional safety precautions can be added to the function (e.g. will the set function overload the variable capacity? If so, don't allow)
- *Getters* will have one return value only and no arguments

```
int getAccNum(){ return acc_number; }
```

- *Setters* have no return values and *usually* take only one argument

```
void setBalance(int val) { balance = val; }
```

Class constructors

- A constructor ALWAYS has the same name as the class name.
- A constructor MUST not have a return type – even void is wrong
- A constructor cannot be independently invoked like other methods in any part of the application – you cannot manually invoke constructors even though they look like methods
- Constructors are invoked only once by class instance creation statements
- Constructors are never inherited

- If no constructor is added, a default constructor is automatically created for you – this default constructor does nothing.
    - *If you create a constructor that does nothing it is also called the default constructor!!*

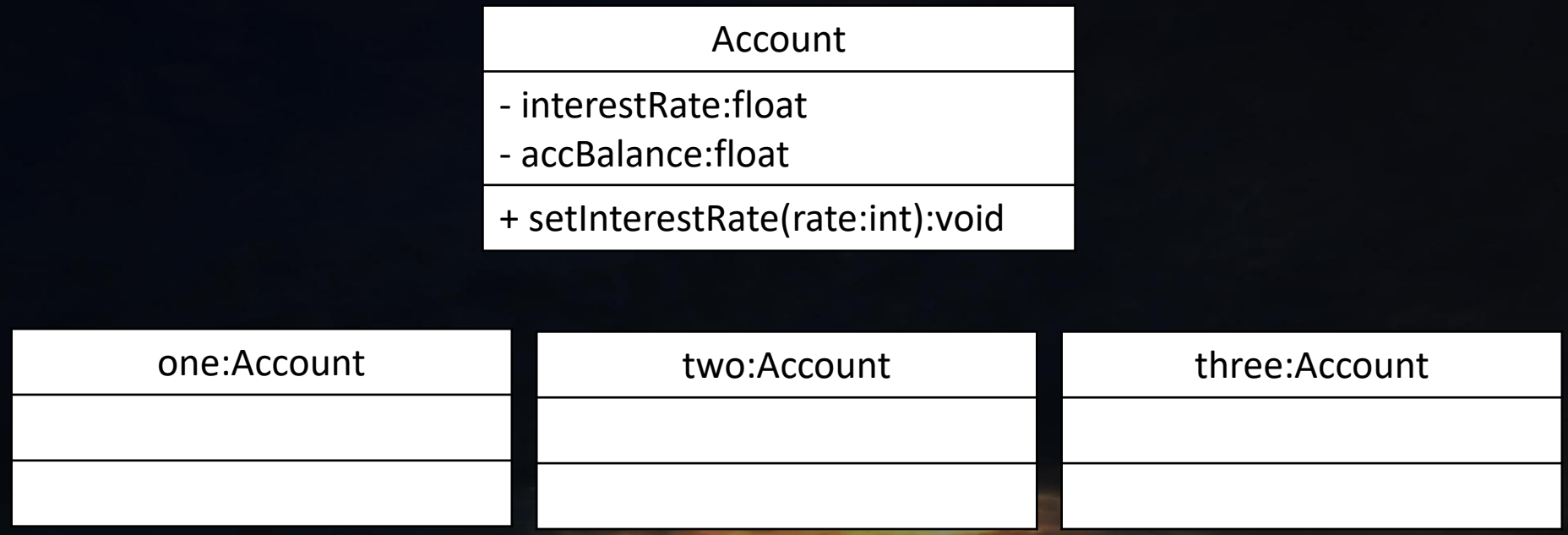
```
public class Account{
   private double balance;
   private int acc_number;
   private String name;
   Account(String s, int num, double amt){
      name = s;
      acc_number = num;     //Variable initialization
      balance = amt;
   }
   Account(){}
}
```

- Which lines are the constructor(s)?
- Which constructor will these statements call during object instantiation?

```
Account acc1 = new Account();
Account acc2 = new Account("Sally",12345,1000000);
Account acc3 = new Account("Will");
```

Static keyword

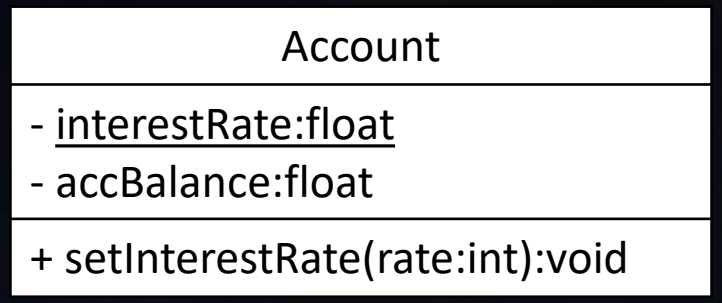
- Similar to `static` in C++
- Allows multiple object instances to share class members (must be from same class)
- For variables, when the first object is created, all static data is initialized to zero (either auto or thru constructor) – any further objects instantiated from the same class will only refer to that *first* object and not create new ones.
- Static methods are methods that exist outside the class and uses no instance variables (i.e. don't 'belong' to a class even though declared within)
- Static methods can only access static variables.
- Non-static methods can access static variables

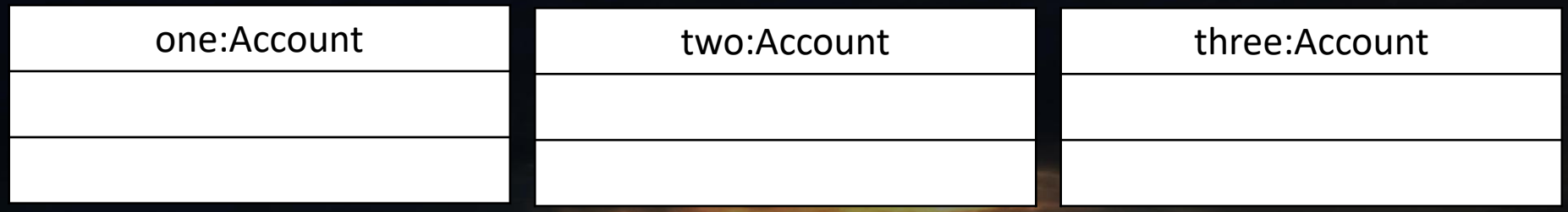


- Example
  The bank has changed the interest rate of all accounts to 10%. How to update this information to all accounts?

```
one.setInterestRate(10);
two.setInterestRate(10);
three.setInterestRate(10);
```

- What if you had one million accounts?

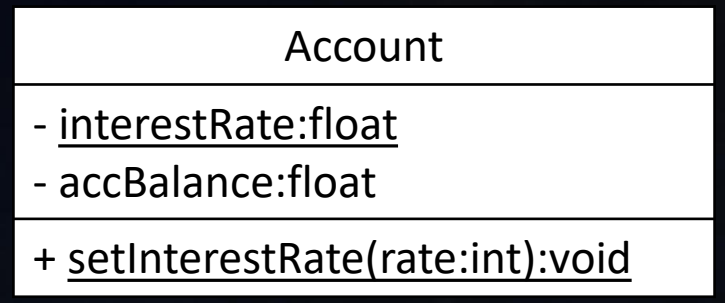


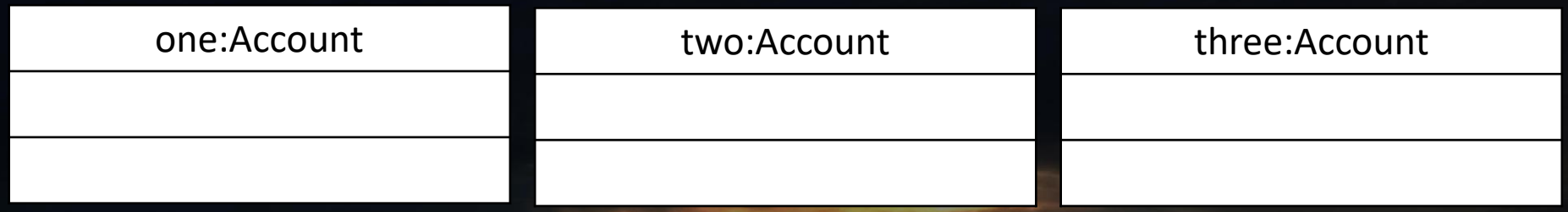


- Example
  The bank has changed the interest rate of all accounts to 10%. How to update this information to all accounts?

```
one.setInterestRate(10);
```

- Setting one object is enough because other objects from the same class share the static variable *interestRate*
- We still need an object instance (examples uses '**One**') because the method used is non-static





- If the **`setInterestRate()`** function was also made static, then object instances are not even required!

```
Account.setInterestRate(10);
```

- Since the method **`setInterestRate( )`** is now static, we no longer need an object instance to call the method.

Static example

```
public class Student {
	private static int count = 0;
	private int id;

	public Student() {
		count = count + 10;
		id = count;
	}
	public int getId() { return id; }
}
```

```
public class Application {

	public static void main(String[] args) {
	   Student stud1 = new Student();
	   Student stud2 = new Student();

	   System.out.println("Student 2 ID = " + stud2.getId());
	}
}
```

1. Which file do you compile?
2. How many methods does the Student class have?
3. How many Student objects are being instantiated?
4. What is/are the object name(s)?
5. What is the output of the program when it runs?
6. Can the statement `Student.getId()` be used in Application? Why?
7. Can the method `getId` in Student be made static?

Constants

- A constant is an identifier that is similar to a variable except that it holds the same value during its entire existence and cannot be changed
- The compiler will issue an error if you try to change the value of a constant
- In Java, we use the **`final`** modifier to declare a constant

```
final int MIN_HEIGHT = 69;
```

- In C++, either **`#define`** or **`const`** was used → these are not valid in Java.
- Note that the keyword '*final*' also affects inheritance of methods (but does not affect attributes) such that any subclass cannot override inherited methods.

Data type conversion

- Conversions can occur two ways and must be handled carefully to avoid losing information
  - Widening conversions are safest because they tend to go from a small data type to a larger one (such as a short to an int)
  - Narrowing conversions can lose information because they tend to go from a large data type to a smaller one (such as an float to an int)
- In Java, data conversions can occur in three ways:
  - assignment conversion
  - promotion
  - casting

Assignment conversion

- Occurs when the value of one data type is assigned to another data type
- Example: If money is a float variable and dollars is an int variable, the following assignment converts the value in dollars to a float

```
money = dollars
```

- Only widening conversions can happen via assignment
- Note that the value or type of dollars did not change

Data promotion

- Also called conversion
- Promotion happens automatically when operators in expressions convert their operands
- For example, if **`sum`** is a **`float`** and **`count`** is an **`int`**, the value of **`count`** is converted to a floating point value to perform the following calculation:

```
result = sum / count;
```

- Casting is the most powerful, and dangerous, technique for conversion – it is an intentional conversion intended by the programmer.
- Both widening and narrowing conversions can be accomplished by explicitly casting a value
- To cast, the type is put in parentheses in front of the value being converted
- For example, if **`total`** and **`count`** are **`integers`**, but we want a floating point result when dividing them, we can cast total:

```
result = (float) total / count;
```

Java ‘built-in’ functions

- Java has a large library of built-in functions for basic operations
- These functions (or methods) belong to existing classes in the Java library

- You have already seen the `println`/`print` earlier – similar to `printf` in C++
- The display functions belong to the `System` class library which is loaded by default by all Java programs
- For other functions (methods) you need to add the corresponding class/package/library to your program – similar to `#include` in C++
- Java has no ‘include’ and instead uses `import`

User input (command-line)

- In C++ this was done using **`scanf`** (or **`getch`**)
- Java uses a **`Scanner`** object to replace **`scanf`**
- **`Scanner`** object comes from the **`util`** library – so you need to add the header declaration

```
import java.util.Scanner;
```

OR

```
import java.util.*;
```

- The **`Scanner`** class provides convenient methods for reading input values of various types
- It can be set up to read input from various sources, including the user typing values on the keyboard
- Keyboard input is represented by the **`System.in`** object

Taking user input

- The following line creates a **`Scanner`** object that reads from the keyboard:

```
Scanner scan = new Scanner (System.in);
```

- The **`new`** operator creates the **`Scanner`** object
- Once created, the **`Scanner`** object can be used to invoke various input methods, such as:

```
answer = scan.nextLine();
```

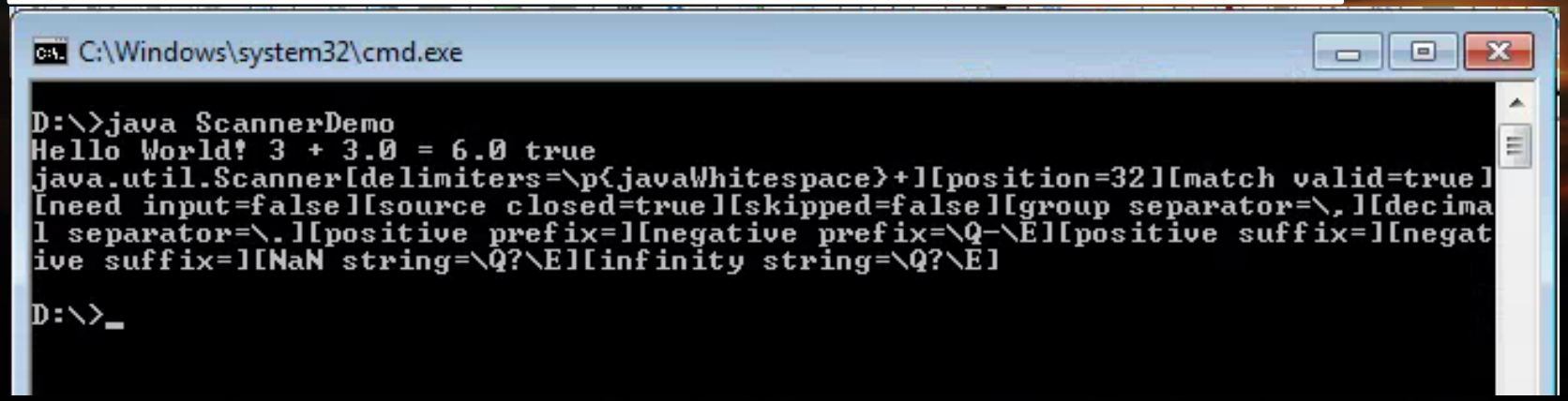
- **`nextLine`** (notice case!) reads the entire line up to return character
- Other options

```
nextInt()
nextLong()
next(regex)
close()
nextDouble()
```

- Reference:
  http://docs.oracle.com/javase/7/docs/api/java/util/Scanner.html

- **`Scanner`** object also has additional methods to support functions such as data type translation
- Example: **`toString()`** function

```
import java.util.*;
public class ScannerDemo {
   public static void main(String[] args) {
      String s = "Hello World! 3 + 3.0 = 6.0 true ";
      Scanner scanner = new Scanner(s);
      System.out.println("" + scanner.nextLine());
      System.out.println(scanner.toString());
      scanner.close();
   }
}
```



Source:http://www.tutorialspoint.com/java/util/scanner_tostring.htm

Example usage of Scanner

```
import java.util.Scanner;
public class GasMileage
{
   //  Calculates fuel efficiency based on user entered values
   public static void main (String[] args)
   {
      int miles;
      double gallons, mpg;

      Scanner scan = new Scanner (System.in);

      System.out.print ("Enter the number of miles: ");
      miles = scan.nextInt();

      System.out.print ("Enter the gallons of fuel used: ");
      gallons = scan.nextDouble();

      mpg = miles / gallons;
      System.out.println ("Miles Per Gallon: " + mpg);
   }
}
```

Example 2

- Load text file, read line-by-line and display on screen

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class ScanExample2{
        public static void main(String[] args) {
                File file = new File("longstory.txt");
                try{
                Scanner sc = new Scanner(file);
                 while (sc.hasNextLine()) {
                     int i = sc.nextInt();
                     System.out.println(i);
                 }
                 sc.close();
                }
                catch(FileNotFoundException e){
                e.printStackTrace();}
        }
}
```

Java classes : Math

- In the package **`java.lang.`**
- Contains some constants and useful mathematical functions.
- All values and methods are declared as static.

```
Math.sin(x);    //sin(x):  x in radians
Math.cos(x);    //cos(x):  x in radians
Math.asin(x);   //sin-1(x)
Math.abs(x);    //absolute value of x
Math.pow(x,y);   //x to the power of y
Math.sqrt(x);   //square-root of x
Math.random();   //a random number between 0 and 1.
```

- *http://docs.oracle.com/javase/7/docs/api/java/lang/Math.html*

Math example

```
public class MathLibraryExample {

  public static void main(String[] args) {

    int i = 7;
    double x = 72.3;
    double y = 0.34;

       System.out.println("|" + i + "| is " + Math.abs(i));
       System.out.println(x+" is approximately "+Math.round(x));
       System.out.println("Ceiling of "+y+ " is "+Math.ceil(y));
       System.out.println("Pi is " + Math.PI);
       System.out.println("e is " + Math.E);
       double angle = 45.0 * 2.0 * Math.PI/360.0;
       System.out.println("cos(" + angle + ") is " +
                                              Math.cos(angle));
       System.out.println("sin(" + angle + ") is " +
                                              Math.sin(angle));
       }
}
```

Java classes : Font

- In the package **`java.awt.`**
- Purpose: Defining the font for texts and labels in Java applications with graphical interfaces
- Font type: TimesRoman, Helvetica, Courier, DialogInput, Symbol. etc
- Font attribute: Font.BOLD, Font.ITALIC, Font.PLAIN etc
- Font size: integer values

- *http://docs.oracle.com/javase/7/docs/api/java/awt/Font.html*

Java classes : Color

- A color in a Java program is represented as an object created from the **`Color`** class
- The **`Color`** class also contains several predefined colors, including the following:

Object	RGB Value
`Color.black`	`0, 0, 0`
`Color.blue`	`0, 0, 255`
`Color.cyan`	`0, 255, 255`
`Color.orange`	`255, 200, 0`
`Color.white`	`255, 255, 255`
`Color.yellow`	`255, 255, 0`

- *http://docs.oracle.com/javase/7/docs/api/java/awt/Color.html*

Java classes : DecimalFormat

- In the package **`java.text`**.
- Purpose: formatting the output of floating point values (e.g. limiting number of decimal points)
- Method format accepts a floating point value and returns a formatted string.

```
import java.text.*;
public class Format{
   public static void main(String[] args){
    DecimalFormat currency = new DecimalFormat("0.00");
    double amount=3.5;
    System.out.println("Amount = $" + amount);
    System.out.println("Amount = $" + currency.format(amount));
   }
}
```

```
Amount = $3.5
Amount = $3.50
```

Java classes : Image

- Java's 2D Image library supports functions to load basic images and display them
- Supports JPEG, GIF, ICO, BMP, PNG, BMP and WBMP.
- Two libraries: **`java.awt.Image`** and **`java.awt.image.BufferedImage`**
- **`BufferedImage`** is a *subclass* of **`Image`** – it will work with **`Graphics`** and **`Graphics2D`** methods that accept **`Image`** objects as parameters
- A **`BufferedImage`** is essentially an **`Image`** object with an accessible data buffer – more efficient and more flexible in terms of image content access

Image class

- Creates Image objects

```
Image getImage(URL urlobj);
Image getImage(URL urlobj, String strobj);
```

- Example 1

```
Image img;
img = getImage(url, "scenery.gif");
```

- Example 2

```
Image img = getToolkit().getImage(url)
```

```
import java.net.*;
import java.awt.*;
import javax.swing.*;
public class QueenApl extends JFrame{
  Image card;
  public static void main(String[] args){
    QueenApl f = new QueenApl();
    f.show();     }
  public QueenApl(){
    setSize(400,400);
    try{
      URL u = new URL("http://img3.16pic.com/00/44/15/16pic_4415600_s.jpg");
      card=getToolkit().getImage(u);
    }catch(MalformedURLException e){
    System.out.println("Error in URL!"); System.exit(1);
}   }
   public void paint(Graphics g){
      g.drawImage(card,120,50,this);
 }    }
```

- **`getToolkit()`** is part of the AWT library for applications.
- Notice how applications also use the same Graphics object in **`paint`** to draw the image onto the screen.

BufferedImage class

- Preferred method of image handling due to better efficiency and more support for image manipulation
- Makes use of additional ImageIO library to load images into BufferedImage object
- So, it requires `java.awt.image.*` and `jawa.imageio.*`
- Example

```
BufferedImage img;
img = ImageIO.read(new File("strawberry.jpg"));
```

- ImageIO decodes File as jpeg format and assigns to img so it can be manipulated by Java2D library.
- To draw the image, it uses the same methods as before from Image

```
public void paint(Graphics g) {
        g.drawImage(img, 0, 0, null);
    }
```

- There are various methods of obtaining images (e.g. inputstream from cameras, video feeds)
- *http://docs.oracle.com/javase/7/docs/api/java/awt/Image.html*

- It is also possible to create a blank Image object as a pixel data buffer (to create a drawing program perhaps?)

```
BufferedImage blnkImage =
  new BufferedImage(100, 50,
                    BufferedImage.TYPE_INT_ARGB);
```

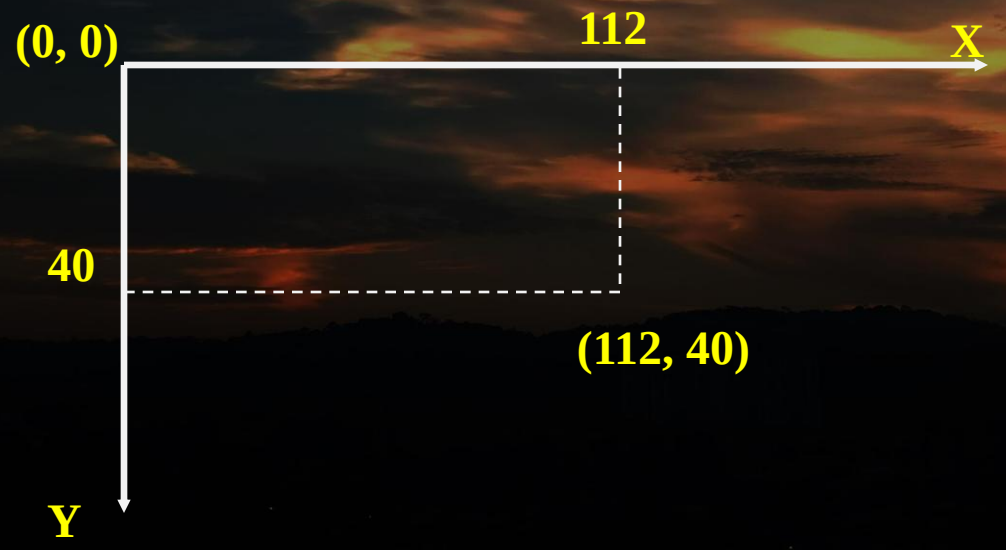
- Drawing the image also has method variants
- *http://docs.oracle.com/javase/7/docs/api/java/awt/Graphics.html*

The Graphics object

- "`Graphics g`" has been seen a few times – what is it?
- Here `g` is an instance of the `Graphics` class - provides the framework for all graphics operations within the AWT (abstract windowing toolkit)
- Requires `java.awt.*` library
- Two functions
    - Create a graphics context
      – an environment to draw/place/move/edit objects graphically
      – include background, foreground, font, color, dimension, output destination (file or screen)
    - Provides functions for basic drawing
      – shapes, text and images to screen
- Has two methods – `update` and `paint` (and also additionally `repaint()`)

Graphics coordinate system

- Graphics in Java are manipulated using coordinate system where every object is referenced through its X and Y values
- Graphics are also made up of pixels – hence resolution of images (and relative size) is referenced by pixel length instead of cm or m or real world measurements.
- Each pixel can be identified using a two-dimensional coordinate system with the origin in the top-left corner of the screen



Java Point object

- Provided in the package java.awt. to provide easier manipulation of coordinates in Graphics object.
- Methods:
  - `getX, getY` - to get the current values of x, y
  - `move`, `translate` – to transpose/move/rotate points
  - `setLocation(X,Y)` / `setLocation(Point)`
- Create Point

```
Point p1 = new Point(3,5);
```

- Combine with Graphics object

Instead of

```
g2.draw(new Line2D.Float(x1, y1, x2, y2));
```

Use

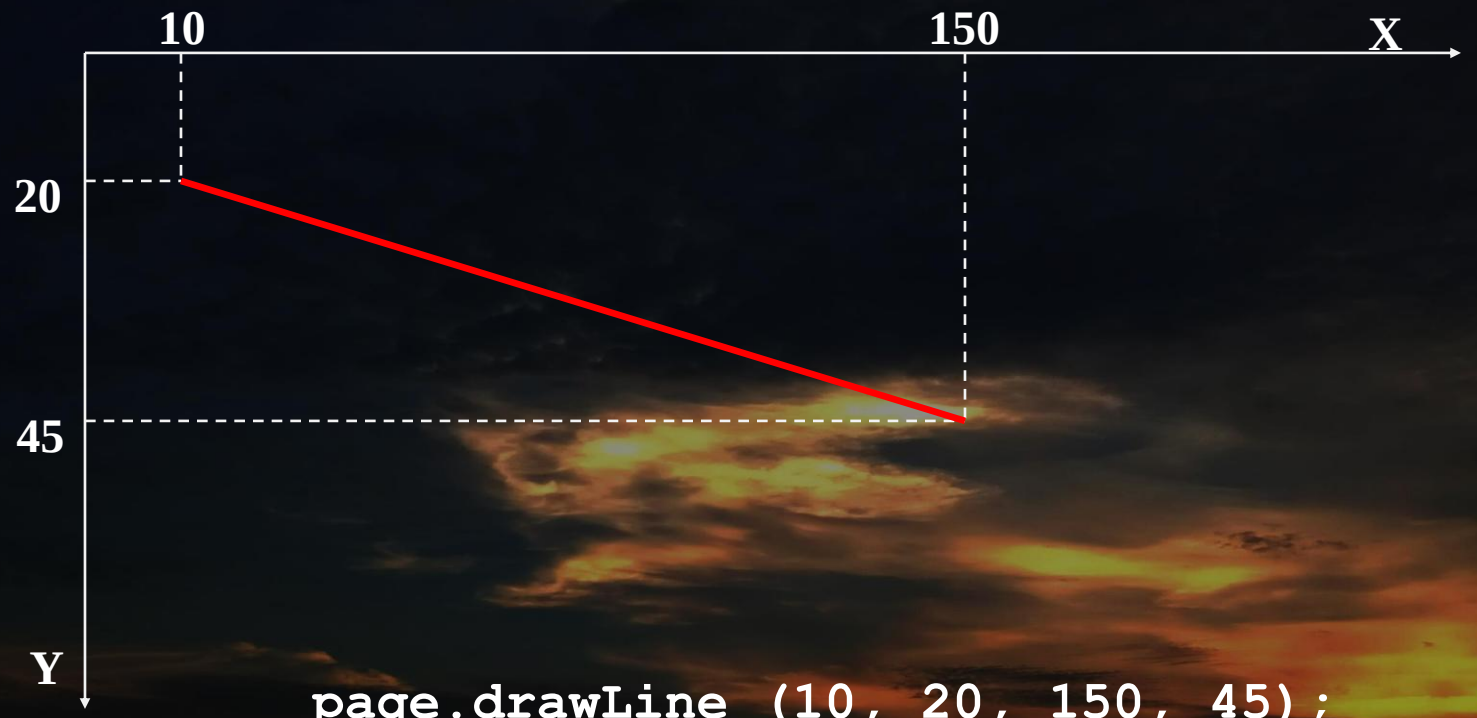
```
g2.draw(new Line2D.Float(Point2D p1, Point2D p2));
```

- *http://docs.oracle.com/javase/7/docs/api/java/awt/Point.html*

Drawing basic shapes

- Makes use of `awt` library
- Methods from the `Graphics` class can be combined to draw shapes in more detail
- A shape can be filled or unfilled, depending on which method is invoked
- The method parameters specify coordinates and sizes
- Shapes with curves, like an oval, are usually drawn by specifying the shape's bounding rectangle (boundary)
- An arc can be thought of as a section of an oval

Basic line

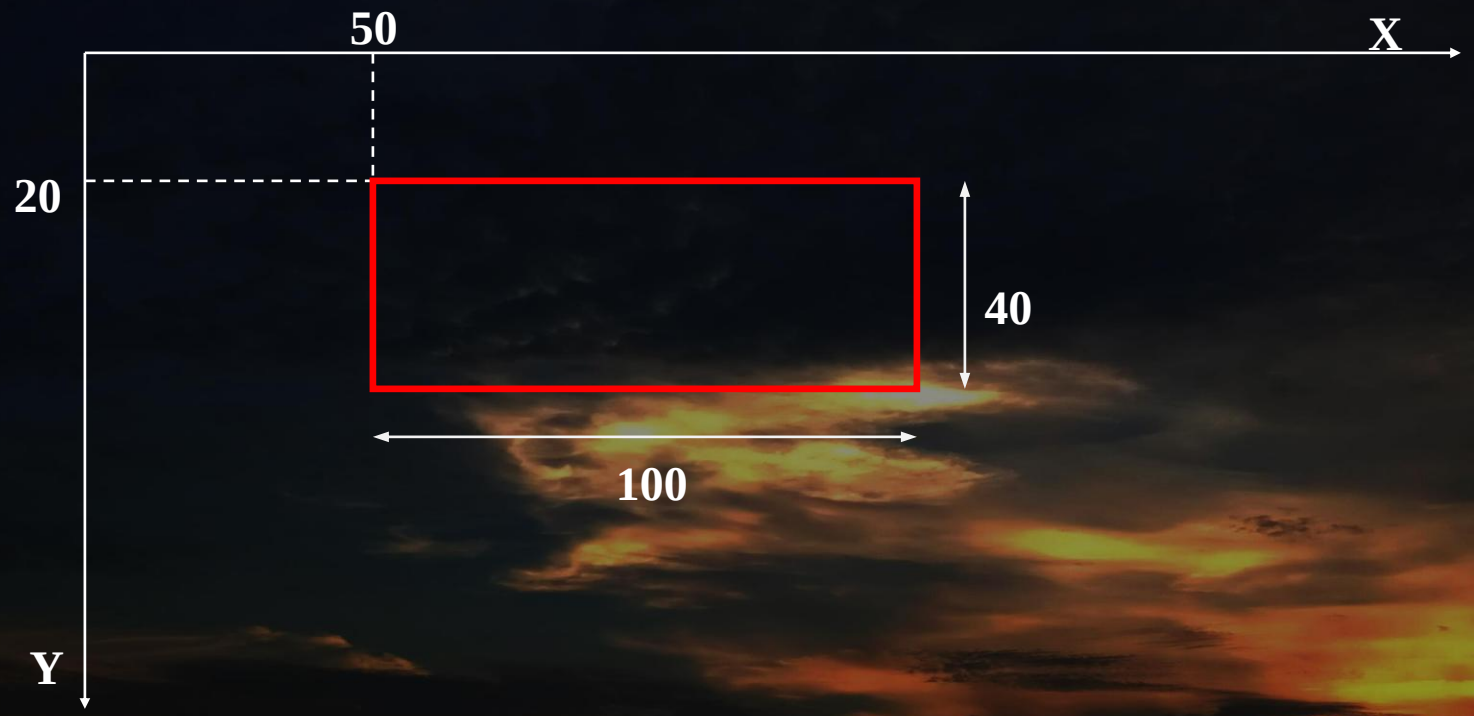



```
page.drawLine (10, 20, 150, 45);
```

or

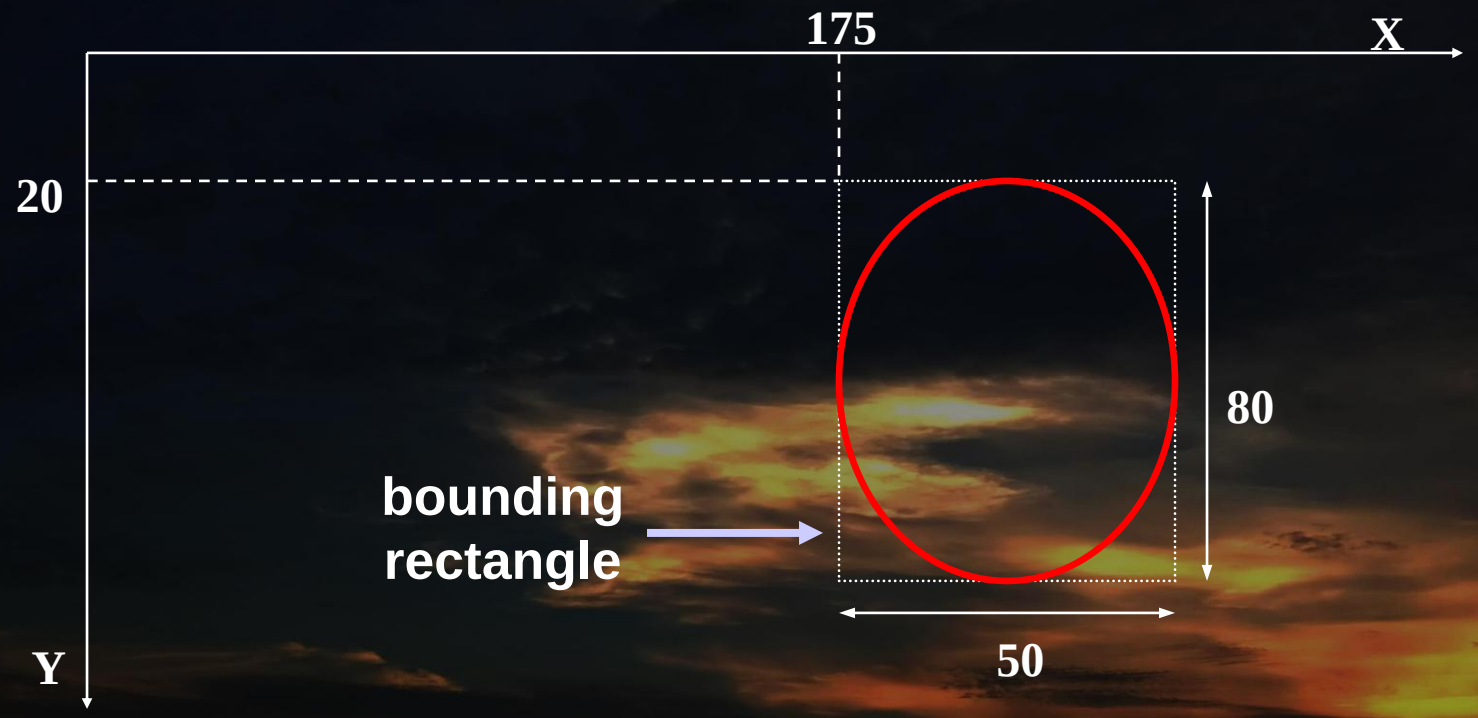
```
page.drawLine (150, 45, 10, 20);
```

Rectangle




```
page.drawRect (50, 20, 100, 40);
```

Oval




```
page.drawOval (175, 20, 50, 80);
```

Graphic properties

- Every drawing surface has a background color
- Every graphics context has a current foreground color
- Both can be set explicitly
- Certain `Graphic` methods have support for other objects like `Font` (`drawString`)

- Combining Graphics methods the following example is possible

Source: Java Software Solutions Foundations of Program Design

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class Snowman extends JFrame
{
    public Snowman()
    {   super("Snowman");
        setSize(305,230);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }
    public static void main(String[] args)
    {   JFrame nFrame = new Snowman();
        nFrame.setVisible(true);
    }
```

```
public void paint (Graphics page)
{   final int MID = 160;
    final int TOP = 60;
    page.setColor (Color.cyan);
    page.fillRect (0,0,300,200);
    page.setColor (Color.blue);
    page.fillRect (0, 175, 300, 50);
    page.setColor (Color.yellow);
    page.fillOval (-40, -20, 80, 80);
    page.setColor (Color.white);
    page.fillOval (MID-20, TOP, 40, 40);
    page.fillOval (MID-35, TOP+35, 70, 50);
    page.fillOval (MID-50, TOP+80, 100, 60);
    page.setColor (Color.black);
    page.fillOval (MID-10, TOP+10, 5, 5);
    page.fillOval (MID+5, TOP+10, 5, 5);
    page.drawArc (MID-10, TOP+20, 20, 10, 190, 160);
    page.drawLine (MID-25, TOP+60, MID-50, TOP+40);
    page.drawLine (MID+25, TOP+60, MID+55, TOP+60);
    page.drawLine (MID-20, TOP+5, MID+20, TOP+5);
    page.fillRect (MID-15, TOP-20, 30, 25);
  }
}
```


Snowman

Other misc classes

- Java Calendar
- *http://docs.oracle.com/javase/7/docs/api/java/util/Calendar.html*

- Java URL
- *http://docs.oracle.com/javase/tutorial/networking/urls/*
- *http://docs.oracle.com/javase/7/docs/api/java/net/URL.html*

- Java Audio
- *https://docs.oracle.com/javase/tutorial/sound/playing.html*

Revision!

- What you need to do on your own
  - Experiment with Java – download the JDK, install and setup your programming environment
  - Look through the information given today – Java has many other features not covered and only the basics were shown (e.g. println has options for text formatting, size, colour etc)
  - Revise on your own about control structures (if-else, case, while, do-while, for loops), logical operators (||, &&, !), mathematical operators and their precedence (+, —, ×, ÷ , %) – those remain the same from C++ and C